

Microservicios

Por Rafik Mas'ad Nasra



Uso de IA generativa

- Se permite utilizar IA generativa en controles y tareas, pero debe ser declarado previamente.
- Todo lo entregado debe poder ser defendido por el o la estudiante en cualquier momento.
- Redacciones poco claras, redundantes o con muletillas poco naturales serán penalizadas.
- El resultado final debe presentarse con estándar profesional: claro, preciso, pertinente y cuidado.

Ustedes aprenderán a...

**Diseñar e implementar un
sistema en una arquitectura
de microservicios.**



En particular:

- Dividir un sistema y sus datos en pequeños y autónomos servicios.
- Implementar distintos métodos de comunicación entre servicios.
- Construir, desplegar y orquestar múltiples servicios de forma automatizada.

- Control de calidad a aplicaciones en microservicios.
- Implementar interfaces de usuario para una aplicación en microservicios.
- Arquitectura evolutiva en el contexto de microservicios.



Unidades

Unidad 1: Introducción a microservicios
Unidad 2: Estrategias de división de servicios
Unidad 3: Comunicación entre servicios
Unidad 4: Implementación
Unidad 5: Despliegue (deployment)
Unidad 6: Interfaz
Unidad 7: Control de Calidad
Unidad 8: Seguridad, resiliencia y escalabilidad
Unidad 9: Arquitectura evolutiva



Para esto tendrán a disposición...

- Estas presentaciones con el resumen de las 9 unidades.
- Material complementario como libros y artículos para profundizar ciertos tópicos.
- Ejemplos funcionales de cada una de las materias.

Unidad 1

Introducción a microservicios

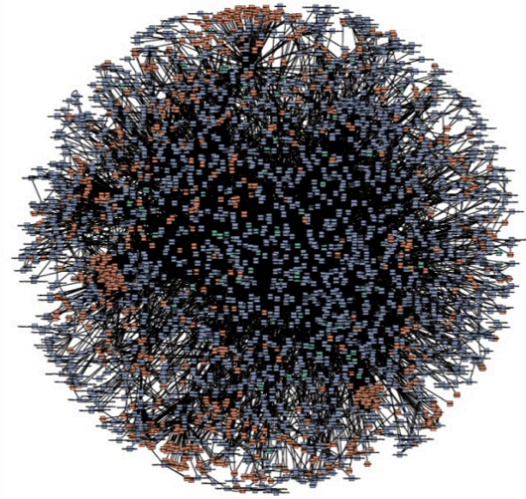
Microservicios es un tema popular...

Y muchas de las grandes empresas de software escriben sus aplicaciones con este patrón arquitectónico.

Netflix, Uber, SoundCloud, Amazon, Spotify y Ebay son algunos ejemplos

✨ Y se pueden lograr hazañas increíbles

- Nuevas versiones cada 3 segundos.
- Coordinar cientos de servicios con latencias de pocos milisegundos dentro del cluster.
- O seguir operando cuando parte de tu aplicación está caída.



Topología de Amazon

**Los microservicios son
pequeños y autónomos
servicios que trabajan en
conjunto a los cuales se
les accede mediante una
red.**

Son pequeños

- "Mantén junto todo lo que cambia por la misma razón, separa las cosas que cambian por diferentes razones". Single Responsibility Principle de Robert C. Martin.
- "... Tan grande como mi cabeza". James Lewis
- "Algo que pueda ser re-escrito en dos semanas". Jon Eaves

- Ser mantenido por un equipo que pueda ser alimentado por dos pizzas (Two Pizza Rule, Amazon).

Veremos más sobre esto en la **Unidad 2**.



Son autónomos

- Ya que... mantener cientos de servicios dependientes entre sí es imposible.
- Idealmente... ningún servicio depende de otro para su funcionamiento (mínimamente acoplados). La autonomía es a nivel de lógica y datos.

- Lo que implica que... **los micro-servicios deben poder actualizarse y desplegarse independientemente.**

Veremos más sobre esto en la **Unidad 2** y la **Unidad 3**.



Trabajan en conjunto

- En aplicaciones como Amazon, Netflix o Uber, en una sesión de un usuario se usan decenas (a veces cientos) de servicios.
- No es factible que la interfaz interactúe con cientos de servicios, por eso se utiliza una puerta de entrada a los otros servicios.

- La comunicación entre los micro-servicios, habitualmente es asíncrona.

Veremos más sobre esto en la **Unidad 3**, **Unidad 4** y la **Unidad 5**.



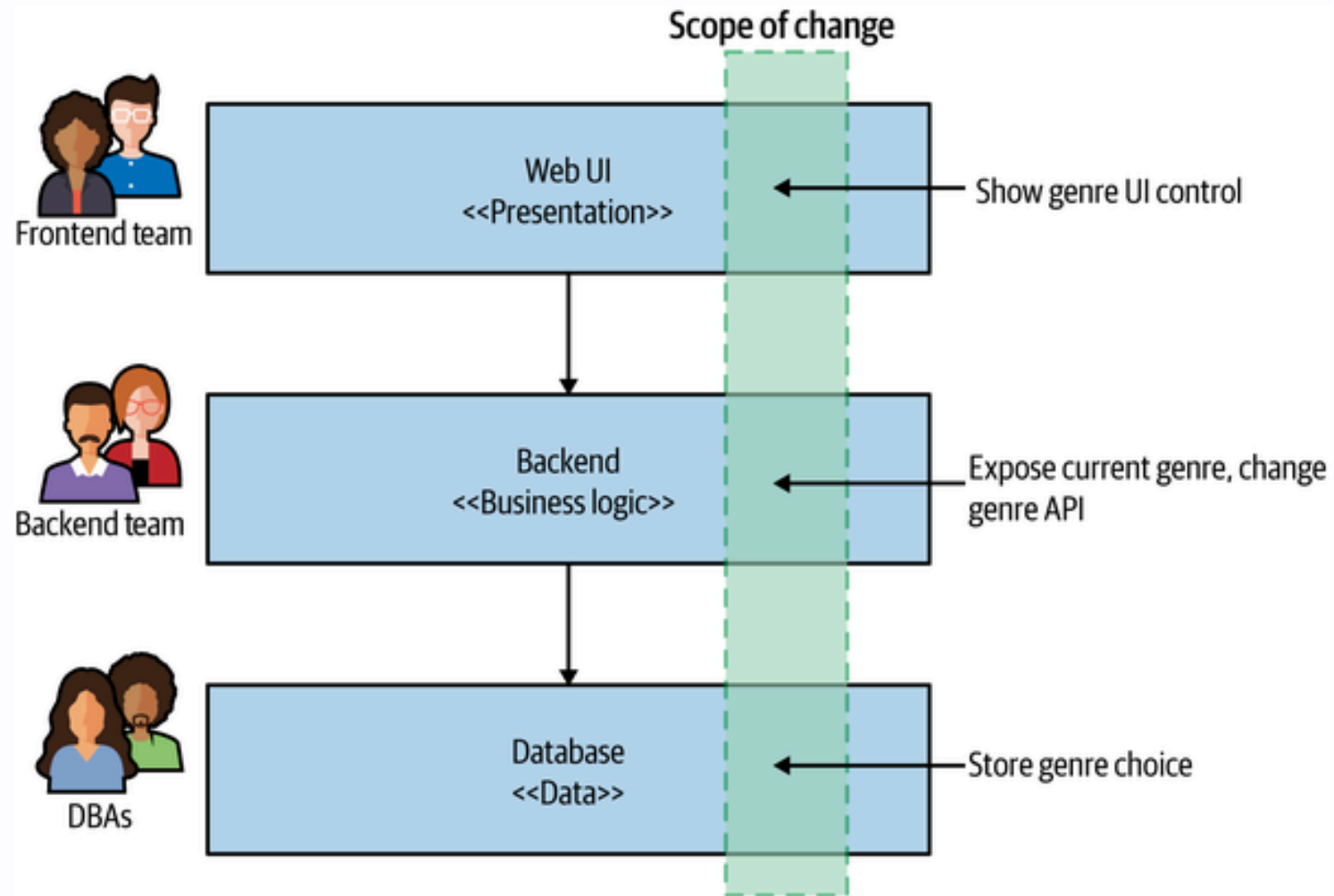
Se les accede mediante una red

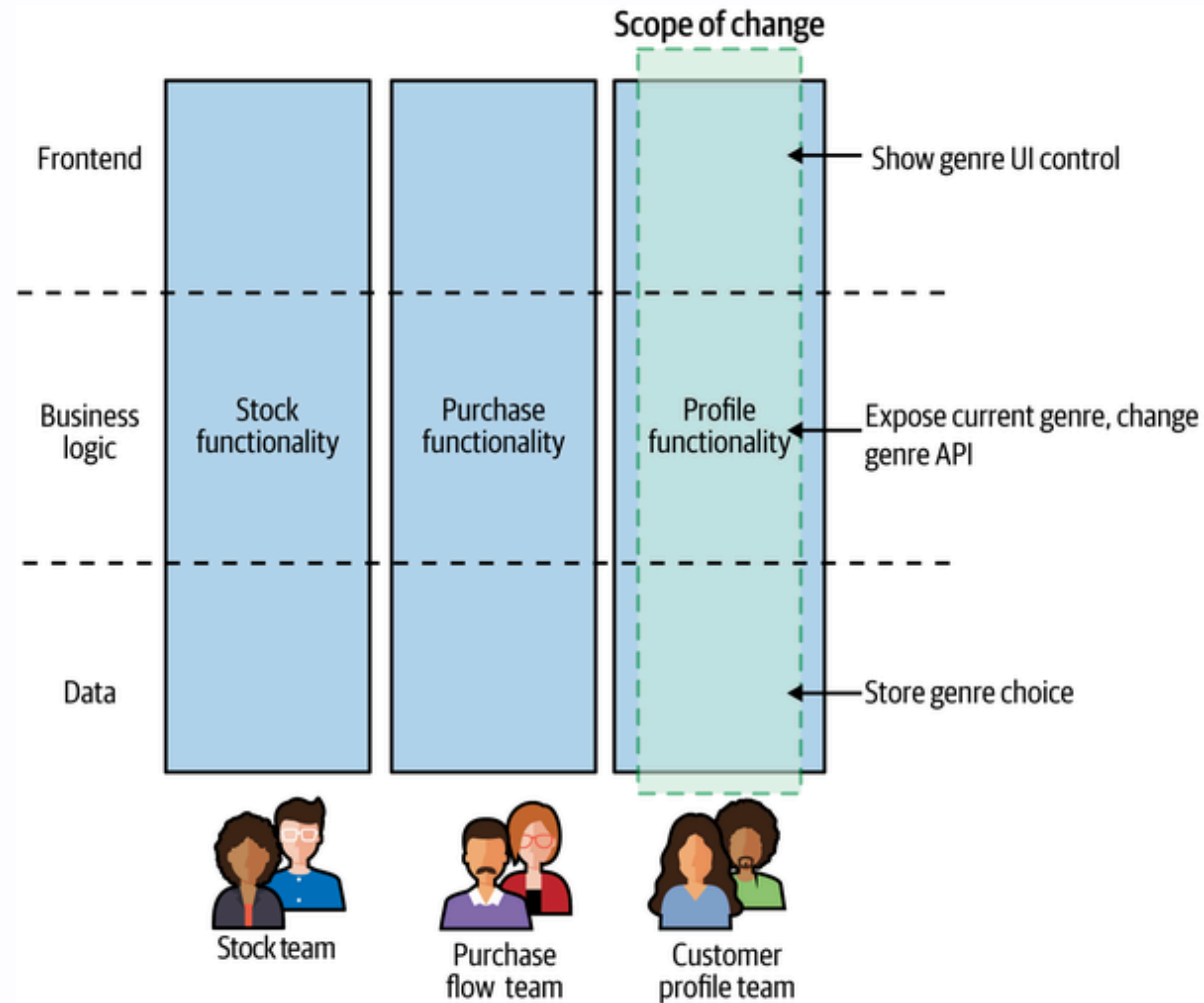
- Se utilizan distintos protocolos y tecnologías para comunicarse con un microservicio: *RESTFul*, *gRPC*, *GraphQL*, *AMQP*, etc.
- La comunicación (*casi siempre*) entre los servicios es mediante la intranet de un *cluster*.

- Estos servicios están dentro de contenedores (como **Docker**) y se despliegan mediante **Kubernetes**.

Veremos más sobre esto en la **Unidad 3** y la **Unidad 5**.

Un sistema en microservicios tiene ventajas sobre uno monolítico: reutilización de componentes, heterogeneidad de tecnologías, alineamiento organizacional, facilidad de despliegue y escalamiento.





Pero también hay desventajas: se requiere una mayor experiencia del equipo, sobrecarga de tecnologías, mayores costos de desarrollo y QA, datos disgregados/inconsistentes y mayor latencia.

Microservicios es recomendable cuando ...

- La aplicación es suficientemente grande para no lograr ser mantenible.
- La aplicación necesita ser desarrollada por grandes equipos.
- Se tienen varios servicios ya desarrollados.
- Se requiere trabajar con múltiples tecnologías.

Microservicios **no** son recomendables cuando...

- El sistema aún no define bien el dominio del problema (*startups*).
- Aplicaciones o equipos de desarrollo pequeños.
- Aplicaciones en que los usuarios/clientes deben realizar el despliegue.

- De hecho, en los últimos años varias empresas han vuelto a monolitos modulares cuando los microservicios no se justificaban. Un caso conocido es el equipo de monitoreo de Prime Video (2023), que redujo costos consolidando sus servicios en un solo proceso.
- La arquitectura se elige según el problema, no según la moda.

La arquitectura de microservicios es la evolución de SOA.

A principios de los 90 no existían una serie de tecnologías que permiten hoy desarrollar microservicios.

🔗 Contenedores

- Es deseado poder aislar los micro-servicios.
- Virtualización es un mecanismo habitual para aislar servicios, pero engorroso y excesivo en microservicios.
- Los contenedores no son máquinas virtuales: aíslan procesos a nivel del kernel de Linux (*namespaces* y *cgroups*), sin virtualizar hardware.

- En la práctica: todos los contenedores comparten el kernel del host, por eso parten en segundos y consumen mucho menos recursos que una VM.
- Tecnologías como **Docker** permiten crear contenedores con un archivo con instrucciones (`dockerfile`).

Ejemplo de `dockerfile`

```
FROM python:3.10

WORKDIR /code

COPY ./app /code/app
COPY ./requirements.txt /code/requirements.txt

RUN pip install --no-cache-dir --upgrade -r /code/requirements.txt

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0",
    "--port", "80", "--reload"]
```

`--reload` reinicia el servidor con cada cambio en el código: úsenlo solo en desarrollo, nunca en producción.



Orquestación de contenedores

- Se necesita orquestar el despliegue de los múltiples microservicios en, potencialmente, múltiples máquinas.
- Para realizar esto, habitualmente se utiliza **Kubernetes**. Para el desarrollo, habitualmente se utiliza **docker-compose** o **minikube**.

- Tanto `Kubernetes`, `docker-compose` y `minikube` utilizan `yaml` para escribir sus archivos de configuración. `yaml` es un lenguaje *human-friendly* de serialización de datos.
- Veremos más sobre esto en la **Unidad 5**.

🧩 Ejemplo de `docker-compose.yml`

```
services:
  demo_01_service_01:
    build: .
    ports:
      - "5000:80"
  demo_01_service_01_mongodb:
    image: mongo:5.0
    volumes:
      - demo_01_service_01_mongodb_container:/data/db
  ...
```

Agregación de *logs*

- A medida que la cantidad de servicios aumenta, es más difícil tener trazabilidad en el sistema. Esto puede ser un problema para detectar (y solucionar) errores en el sistema.
- Los sistemas de agregación de *logs* permiten ver todos los *logs* de todos los sistemas en el mismo lugar.

- Se complementa con patrones como IDs de correlación: un identificador de la transacción/interacción del usuario.

'La' nube (cloud)

La nube facilita (y en muchos casos viabiliza) una arquitectura de microservicios. Se ofrece en distintos niveles de abstracción:

- IaaS (*infrastructure as a service*): máquinas virtuales y redes bajo demanda (EC2).
- CaaS (*containers as a service*): se ejecutan contenedores sin administrar servidores (Cloud Run, ECS).

- PaaS (*platform as a service*): se entrega el código y la plataforma se encarga del resto (Heroku, App Engine).
- SaaS (*software as a service*): software listo para usar (Gmail, Grafana Cloud).

- Permite contratar infraestructura bajo demanda (cobro por uso) lo que facilita escalar microservicios.
- Ofrece software como bases de datos o *message brokers* pre-instalados lo que facilita utilizar diversidad de tecnologías.
- Cluster de Kubernetes administrado por el proveedor lo que facilita la gestión de la infraestructura.

Ejemplo: `./demo_01`

- `Docker` y `docker-compose` en funcionamiento.
- APIs en `FastAPI` donde un servicio realiza consulta a otro. Documentación de las APIs en `localhost:$puerto/docs`.
- El problema del n+1 (solución en la **Unidad 4**).
- Ejemplos de *logging* en los servicios.



Tarea

Crear un par de (nano) servicios mediante **FastAPI**, que envíen sus *logs* mediante **Promtail** a un *log aggregation system* **Loki** y visualizar dichos *logs* mediante **Grafana**. Todo debe estar desplegado mediante **docker-compose**.



Material complementario

- Building microservices: Designing fine-grained systems, Sam Newman (2021). O'Reilly. Capítulo 1.
- [Microservices, Martin Fowler y James Lewis \(2014\)](#). El artículo que popularizó el término.
- [State of Microservices 2020, The Software House](#)